

COMP101

Tutorial-08

Decision Tables/Trees

No task associates with this tutorial

End of Tutorials

Decision Tree: Example

- Business Rule for an ATM:

‘Cash withdrawal approved if user has enough funds or has credit’

Conditions	R1	R2	R3
Withdrawal Amount <= Balance	T	F	F
Credit granted	-	T	F
Actions			
Withdrawal granted	T	T	F

← rules

Decision Table g for g

- Decision Table is a tabular representation of all conditions and actions
- Used when processing logic is complicated and has multiple conditions
- Main components
 - Conditions Stubs, Action Stubs, and Rules
- 'cause-effect tables'

Types of decision tables G for geeks

- **Limited Entry :**

- condition entries are restricted to binary values (T/F)
- This is our focus on the tutorial

- **Extended Entry :**

- condition entries have more than two values
- Uses multiple conditions where a condition may have many possibilities instead of only 'true' and 'false'

Decision Tables ('cause-effect tables')

- From a problem specification we extract conditions and actions required by the program and construct a decision table
 - Clarifies complex logic in if-then-else statements
 - Decisions based on T/F outcomes ('limited entry table')
- Can check and verify all combinations and conditions
 - Balanced ('complete') when all possible i/o is considered
- Can be represented as a decision tree for very complex scenarios

Decision Table: Advantages

- Visualization of Cause and effect relationships
- Easier to understand and review than reading code
- If too complex, can be readily broken down into simpler tables
- Consistent format and can use semi-standardized language
- Actions can be taken from the summarized outcomes of a situation
- Table users don't need to know how to use a computer

Decision Table: Disadvantages

- Decision tables are not well suited to large-scale applications. There is a requirement of splitting huge tables into smaller ones to eliminate redundancy.
- The complete sequence of actions is not reflected in the decision tables.
- A partial solution is presented.

Decision Tables: Application domain

- The order of rule evaluation has no effect on the resulting action
- Decision tables can be applied easily at the unit level only
- Once a rule is satisfied and the action selected, another rule needs to be examined
- The restrictions do not eliminate many applications

Construction: 1. Problem statement

Customer order:

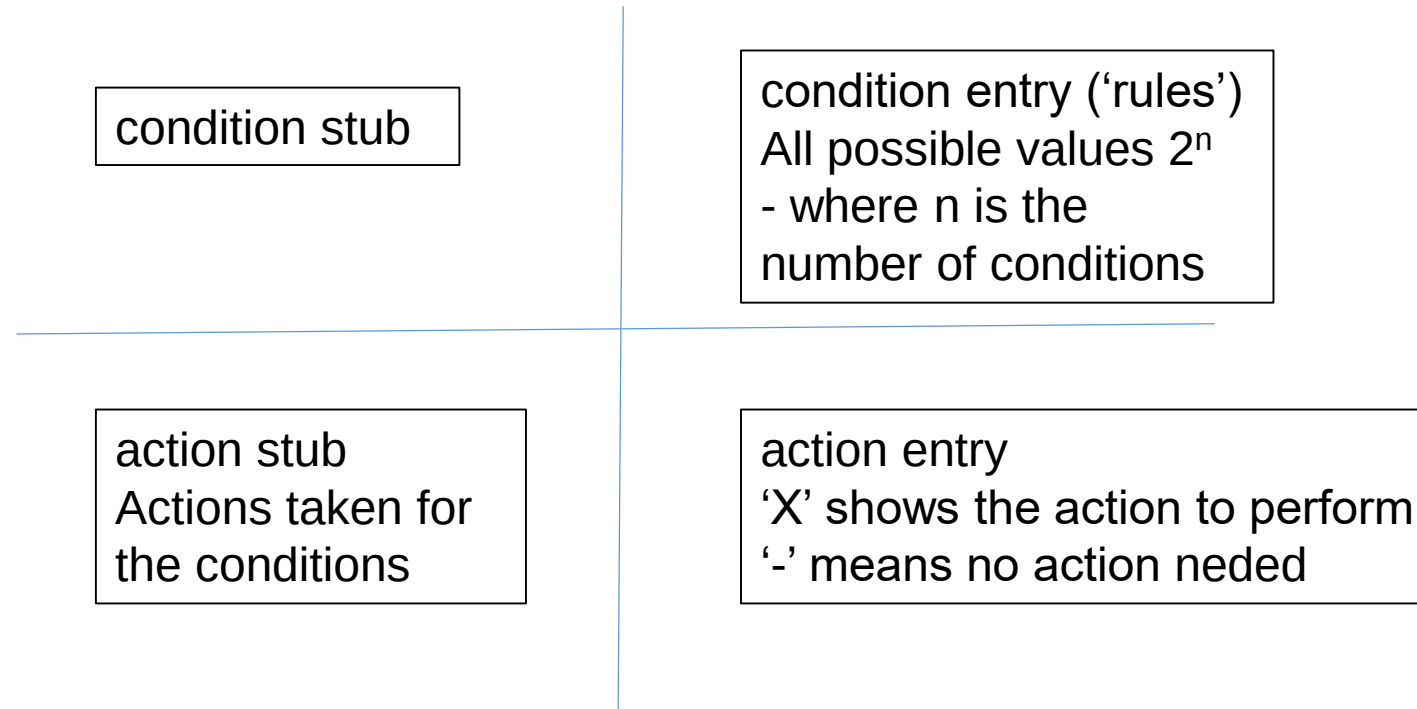
‘A customer order is accepted if the customer credit is okay or if they are a prompt payer, otherwise refer to manager’

- Identify conditions and actions

A customer order is accepted if the customer credit is okay or if they are a prompt payer, otherwise refer to manager

Construction: 2. Build the table

- Construct a quadrant



Construction: 3. Populate the table

- Add the conditions and actions

	R1	R2	R3	R4	
Credit okay	Y	Y	N	N	We have 2 outcomes (Y or N) We have 2 conditions Number of rules = $2^{\text{number of conditions}}$ $= 2^2 = 4$ rules
Prompt payer	Y	N	Y	N	
Accept order	X	X	X	-	'X' means action to be performed '-' means no action needed
Refer	-	-	-	X	

Construction: 3. Apply the halving rule (large tables)

- We need to identify all possible combinations of Y and N for the conditions based on $2^{\text{number of conditions}}$

e.g.

for 2 conditions we need 4 rules $2^2 = 4$

for 3 conditions we need 8 rules $2^3 = 8$

for 4 conditions we need 16 rules $2^4 = 16$

etc

Construction: 3. Apply the halving rule (large tables)

- We need to identify all possible combinations of Y and N for the conditions , e.g. for 3 conditions we need $2^3 = 8$ rules

a) Number the rules

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

b) Put a 'Y' in half of them and a 'N' in the other half

1	2	3	4	5	6	7	8
Y	Y	Y	Y	N	N	N	N

Construction: 3. Apply the halving rule (large tables)

From (b) we have:

1	2	3	4	5	6	7	8
Y	Y	Y	Y	N	N	N	N

c) Now halve the Y and N entries again, i.e. on the top line we have 4 'Y' followed by 4 'N':

On the next line down we put 2 'Y' and 2 'N'

1	2	3	4	5	6	7	8
Y	Y	Y	Y	N	N	N	N
Y	Y	N	N	Y	Y	N	N

Construction: 3. Apply the halving rule (large tables)

From (c) we have:

1	2	3	4	5	6	7	8
Y	Y	Y	Y	N	N	N	N
Y	Y	N	N	Y	Y	N	N

d) Now halve the Y and N entries again, i.e. on the second line we have 2 'Y' and 2 'N' (i.e. half of the top line):

On the next line down we put 1 'Y' and 1 'N' i.e. half of the second line

1	2	3	4	5	6	7	8
Y	Y	Y	Y	N	N	N	N
Y	Y	N	N	Y	Y	N	N
Y	N	Y	N	Y	N	Y	N

Construction: 3. Apply the halving rule (large tables)

From (d) we have:

1	2	3	4	5	6	7	8
Y	Y	Y	Y	N	N	N	N
Y	Y	N	N	Y	Y	N	N
Y	N	Y	N	Y	N	Y	N

e) Rule building is complete – all Y and N combinations are covered

Construction: 3. Apply the halving rule (large tables)

If we had 4 conditions we need 2^4 rules = 16 rules to give:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Y	Y	Y	Y	Y	Y	Y	Y	N	N	N	N	N	N	N	N
Y	Y	Y	Y	N	N	N	N	Y	Y	Y	Y	N	N	N	N
Y	Y	N	N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N

Construction: 4. Remove redundancies (optional)

- When all combinations of condition and actions are done, a large the table can be collapsed by removing redundant rules, e.g.
 - Impossible combinations of actions and conditions
 - Possible, but infeasible combinations
 - Combinations that do not affect the outcome
- The minimum coverage for a decision table is at least one test case per a decision rule
 - Coverage % = $\frac{\text{The number of decision rules tested by at least one test case}}{\text{The total number of decision rules}}$

<https://www.qaoncloud.com/>

Construction: 4. Remove redundancies (optional)

- Business Rule: Login – input is valid, invalid or blank
- Login accepted when id and password are correct
- Error message when any or both are incorrect
- Error message if any input is blank
- This gives us 3 conditions:
 - user inputs something for id – correct or invalid
 - user inputs something for password – correct or invalid
 - User inputs nothing for id or password

Construction: 4. Remove redundancy (optional)

- Add the conditions and actions and check for redundancy

	R1	R2	R3	R4	R5	R6	R7	R8
id valid	Y	Y	Y	Y	N	N	N	N
password valid	Y	Y	N	N	Y	Y	N	N
blank input	Y	N	Y	N	Y	N	Y	N
login allowed	-	X	-	-	-	-	-	-
login refused	-	-	X	X	X	X	X	X

Construction: 4. Remove redundancy (optional)

- Rule1 redundant: User can't input blank if id and password are i/p
- Rule2 okay: id i/p valid, password i/p valid, no blank i/p
- Rule3 redundant: id i/p valid, password i/p invalid, can't input a blank
- Rule4 okay: id i/p valid, password i/p invalid, no blank i/p
- Rule5 okay: id i/p invalid, password i/p valid, blank i/p against id
- Rule6 okay: id i/p invalid, password i/p valid, no blank i/p
- Rule7 okay: id i/p invalid, password i/p invalid, 2 blank i/p
- Rule8 okay: id i/p invalid, password i/p invalid, no blank i/p

Construction: 4. Remove redundancy (optional)

- Revised table (collapsed)

	R2	R4	R5	R6	R7	R8
id valid	Y	Y	N	N	N	N
password valid	Y	N	Y	Y	N	N
blank input	N	N	Y	N	Y	N
login allowed	X	-	-	-	-	-
login refused	-	X	X	X	X	X

Construction: 4. Remove redundancy (optional)

- Technically only Rule2 works, hence all other rules could be removed
- However, using the rules as they are allows us to identify if i/p was made or whether a blank was input (rules 5 and 7)

	R2	R4	R5	R6	R7	R8
id valid	Y	Y	N	N	N	N
password valid	Y	N	Y	Y	N	N
blank input	N	N	Y	N	Y	N
login allowed	X	-	-	-	-	-
login refused	-	X	X	X	X	X

Decision Tree

- A decision tree is a graph that always uses a branching method in order to demonstrate all the possible outcomes of any decision
- Show a better representation of decision outcomes
- Consists of three nodes
 - Decision Nodes, Chance Nodes, and Terminal Nodes
- **Types of the decision tree:**
- Categorical variable decision tree
- Continuous variable decision tree

Decision Tree: Advantages

- A decision tree is simple to comprehend and use.
- New scenarios are simple to add.
- Can be combined with other decision-making methods.
- Handling of both numerical and categorical variables
- The classification does not require many computations.
- Useful in analysing and solving various business problems.

Decision Tree: Disadvantages

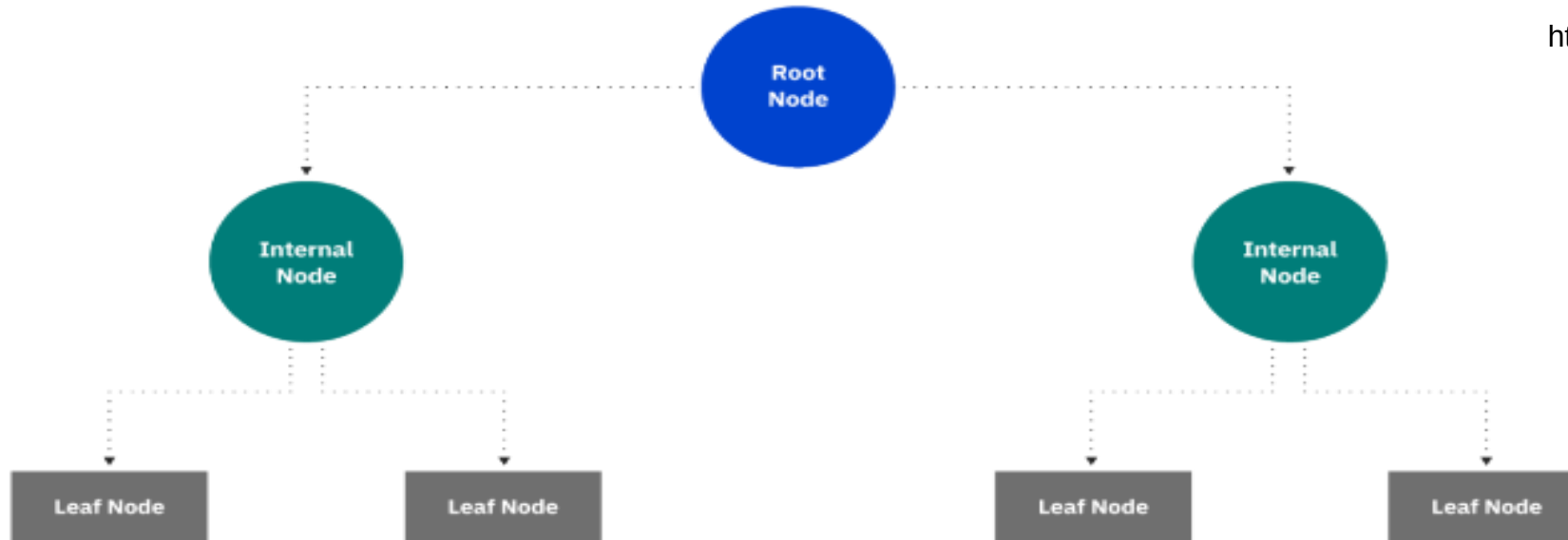
- They are inherently unstable, which means that a slight change in the data can have a result in a change in the structure of the optimal decision tree, and they are frequently wrong.
- These are less suitable for estimation tasks where the outcome required is the value of a continuous variable.
- The alternative options perform better with the same data. A random forest of decision trees can be used as a replacement but it is not as straightforward to comprehend as a single decision tree.
- Calculations can become quite complicated, especially when several values are uncertain and/or multiple outcomes are related.

Decision Table vs Decision Tree

	Decision Table	Decision Tree
1.	Decision Tables are a tabular representation of conditions and actions.	Decision Trees are a graphical representation of every possible outcome of a decision.
2.	We can derive a decision table from the decision tree.	We can not derive a decision tree from the decision table.
3.	It helps to clarify the criteria.	It helps to take into account the possible relevant outcomes of the decision.
4.	In Decision Tables, we can include more than one 'or' condition.	In Decision Trees, we can not include more than one 'or' condition.
5.	It is used when there are small number of properties.	It is used when there are more number of properties.
6.	It is used for simple logic only.	It can be used for complex logic as well.
7.	It is constructed of rows and tables.	It is constructed of branches and nodes.
8.	The goal of using a decision table is the generation of rules for structuring logic on the basis of data entered in the table.	A decision tree's objective is to provide an effective means to visualize and understand a decision's available possibilities and range of possible outcomes.

Decision Tree

<https://www.ibm.com/topics/decision-trees>



Decision tree starts with a root node, which does not have any incoming branches. The outgoing branches from the root node then feed into the internal nodes, also known as decision nodes. Both node types conduct evaluations to form homogenous subsets, which are denoted by leaf nodes, or terminal nodes which represent all the possible outcomes within the dataset.

Decision Tree

<https://www.ibm.com/topics/decision-trees>

